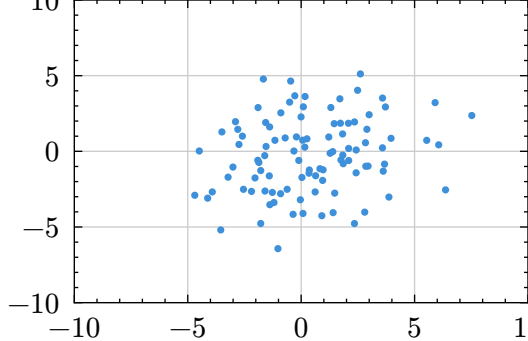
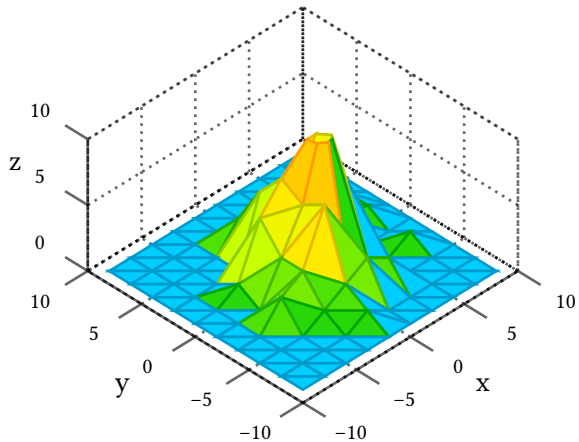


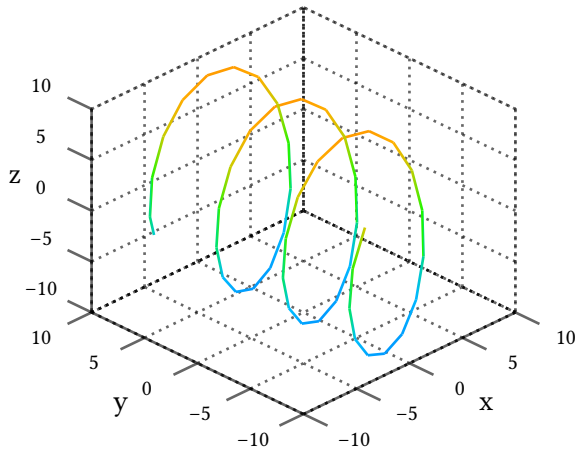
```
#lq.diagram(  
  xlim: (-10, 10),  
  ylim: (-10, 10),  
  lq.scatter(dxs, dys),  
)
```



```
#diagram(  
  zaxis: (lim: (0, 10)),  
  pt.distribution(  
    fill-color-fn: cfn0,  
    stroke-color-fn: cfn0s,  
    mark: none,  
    dxs,  
    dys,  
  ),  
)
```



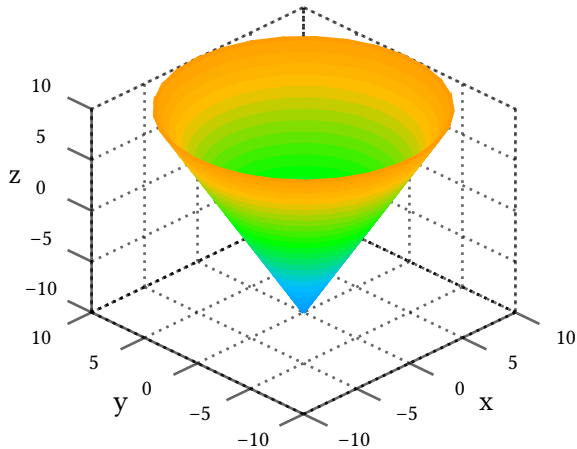
```
#diagram(  
  pt.lineplot(  
    xs.map(x => calc.cos(x) * 5),  
    xs,  
    xs.map(x => calc.sin(x) * 10),  
    stroke-color-fn: cfn,  
  ),  
)
```



```

#let num = 30
#let domain = pt.domain((0, calc.pi),
    (0, 1.75 * calc.pi), v-num: num)
#diagram(
    pt.planeplot(
        domain.map(((u, v)) => u * calc.sin(v) * 5),
        domain.map(((u, v)) => u * calc.cos(v) * 5),
        domain.map(((u, v)) => u * 10 - 10),
        num: num,
        stroke-color-fn: cfn,
        fill-color-fn: cfn,
    ),
)

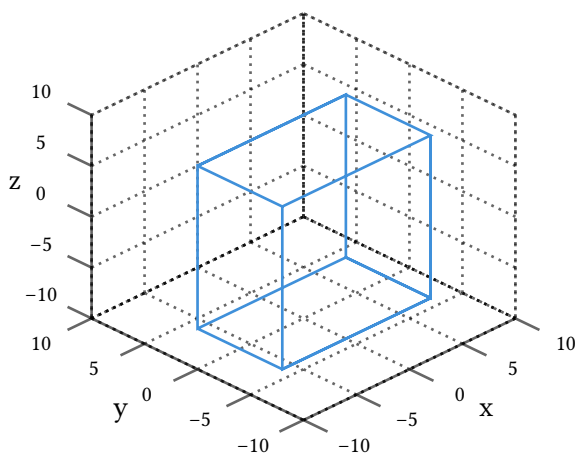
```



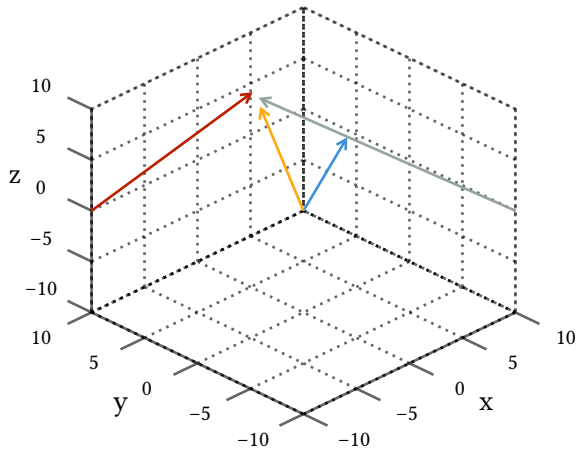
```

#diagram(
  pt.path(
    ..(
      (-1, -1, 1), (1, -1, 1), (1, 1, 1),
      (-1, 1, 1), (1, 1, 1), (1, 1, -1),
      (1, -1, -1), (1, -1, 1), (1, -1, -1),
      (1, -1, -1), (1, 1, -1), (-1, 1, -1),
      (-1, 1, 1), (-1, -1, 1), (-1, -1, -1),
      (1, -1, -1), (-1, -1, -1), (-1, 1, -1),
    ).map(((x, y, z)) => (
      x * 7 - 3, y * 4 - 4, z * 8 + 2)
    ),
  ),
)

```



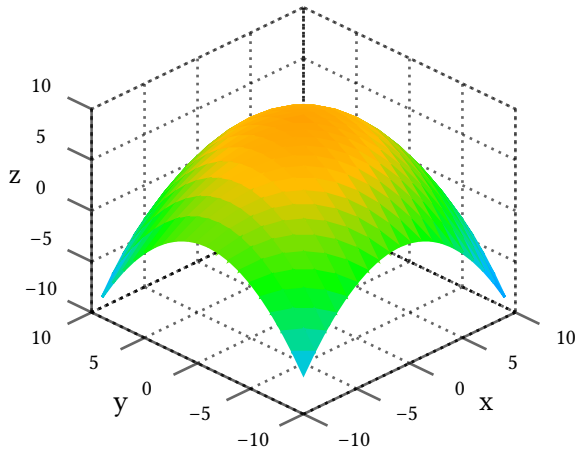
```
#diagram(  
  pt.vec((9, 5, 0)),  
  pt.vec((0, 4, 8)),  
  pt.vec((-10, 10, 0), (0, 5, 9)),  
  pt.vec((10, -10, 0), (0, 4, 9)),  
)
```



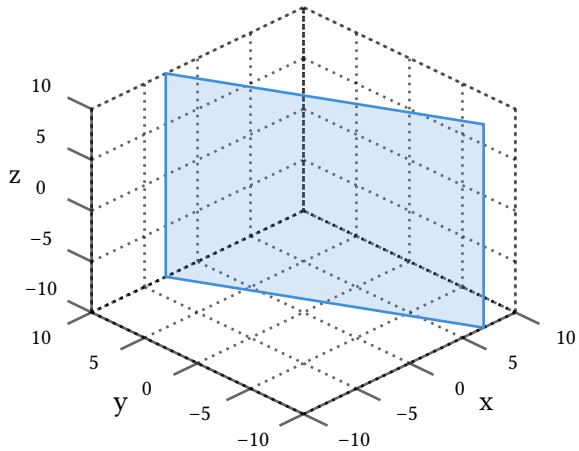
```

#diagram(
  pt.planeparam(
    (x, y) => {
      let z = (1 - x * x - y * y) / 10
      (if z < 0 { z } else { calc.sqrt(z) }
        + 9)
    },
    stroke-color-fn: cfn,
    fill-color-fn: cfn,
    steps: 20,
  ),
)

```



```
#diagram(  
  pt.plane(  
    pt.plane-normal((1, 0.5, 0), 2),  
  ),  
)
```




```
#diagram(  
  pt.line(pt.line-parametric((1, 2, 3), (2, 0, 1))),  
)
```

